

Лекция 8. Централизованный сбор логов

Syslog и rsyslog: от локальных журналов к серверу-коллектору

Цель лекции

- Понять, зачем предприятию нужен централизованный сбор логов и как настроить базовую пересылку журналов через rsyslog.
- Объяснить проблему локальных журналов на нескольких серверах.
- Разобрать базовые элементы Syslog-сообщения.
- Понять роли клиента, релея и сервера-коллектора.
- Настроить log01 для приёма журналов.
- Настроить web01 для отправки журналов.
- Проверить доставку через logger и tail.
- Разобрать типовые ошибки и диагностику.

Учебный сценарий

- Внутренняя учебная зона company.test: один сервер собирает журналы, второй сервер отправляет события.
- log01.company.test — сервер-коллектор.
- IP log01: 192.168.10.100.
- web01.company.test — клиент-отправитель.
- web01 отправляет системные события на log01.
- Удалённые логи сохраняются в
/var/log/remote/%HOSTNAME%/PROGRAMNAME%.log.

Роли в учебном стенде

Узел	Роль	Что делает
log01	Log Collector	Принимает и хранит журналы
web01	Log Generator	Создаёт и отправляет события
rsyslog	Служба логирования	Принимает, фильтрует, записывает или пересылает сообщения
logger	Тестовая утилита	Создаёт контрольное Syslog-сообщение

1. Зачем нужен централизованный сбор логов

Проблема локальных журналов в инфраструктуре

Локальные журналы: что это

- Локальный журнал — файл с событиями, который хранится на том же сервере, где эти события возникли.
- Linux-службы пишут события в `/var/log`.
- Журналы помогают расследовать ошибки и инциденты.
- На одном сервере их можно просматривать вручную.
- В инфраструктуре из многих серверов ручной просмотр становится медленным.
- Администратору нужно единое место для анализа событий.

Почему локальных журналов недостаточно

- Сервер может выйти из строя вместе со своими логами.
- При переполнении диска старые события могут потеряться.
- После взлома злоумышленник может удалить следы из `/var/log`.
- События с разных серверов сложно сопоставлять по времени.
- Администратор тратит время на подключение к каждому узлу.

Что даёт центральный log-сервер

- Журналы с разных серверов собираются в одном месте.
- Удалённые события сохраняются отдельно от локальных логов log01.
- Проще искать события по хосту, службе и времени.
- Сложнее удалить следы атаки только на скомпрометированном web01.
- Можно настроить ротацию и хранение журналов централизованно.

Локальные журналы и централизованный сбор логов

16:9

❌ Проблема: локальные логи

- Журналы распределены по серверам
- Администратор подключается к каждому
- При взломе логи могут быть удалены
- Сложно сопоставлять события по времени
- Перепополнение диска → потеря данных

web01

HTTP сервер
rsyslog клиент

db01

СУБД сервер
rsyslog клиент

app01

Приложение
rsyslog клиент

Сеть

UDP/TCP 514

Syslog

RFC 5424

log01

rsyslog
collector

UDP 514
TCP 514

/var/log
/remote

✅ Решение: централизованный сбор

- Все журналы в одном месте
- Единая структура хранения
- Защита от удаления следов
- Легкий поиск и корреляция
- Ротация и архивация логов

📁 Структура хранения на коллекторе log01

`/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log`

Динамический шаблон RemoteLogs

Пример:

`/var/log/remote/web01/nginx.log`

📄 Компоненты Syslog-сообщения

🔑 Facility

auth, cron, daemon
kern, mail, syslog
local0-local7

⚠️ Severity

0 Emergency → 7 Debug
Error, Warning, Info
Критичность события

🔑 Транспорт

UDP 514 — быстрый
TCP 514 — надёжный
TLS — защищённый

📄 Message

Текст события
Timestamp, Hostname
Понятный администратору

Что должен обеспечить log-сервер

- Принимать события по сети.
- Сохранять журналы в понятной структуре каталогов.
- Отделять удалённые логи от локальных логов самого коллектора.
- Иметь достаточно места на диске.
- Использовать logrotate для ограничения роста журналов.
- Ограничивать доступ, потому что журналы могут содержать чувствительные данные.

Итог раздела 1

- 1 Локальные журналы полезны, но неудобны при большом числе серверов.
- 2 Централизация повышает наблюдаемость и помогает расследовать инциденты.
- 3 Сервер-коллектор должен быть защищён лучше обычного прикладного сервера.
- 4 Централизованный сбор не заменяет защиту узлов, но повышает шанс сохранить события.



2. Syslog: формат, важность и транспорт

Как событие превращается в сообщение журнала

Что такое Syslog

- Syslog — стандартный подход к описанию и передаче системных сообщений журналирования.
- Syslog можно понимать как формат сообщения.
- Syslog также описывает сетевую передачу журналов.
- Сообщение содержит источник, важность, время, имя узла и текст.
- Разные Linux-серверы и сетевые устройства могут отправлять события в общем виде.
- В Linux часто используется rsyslog как реализация Syslog.

Из чего состоит Syslog-сообщение

- Facility — категория или источник события.
- Severity — уровень важности события.
- Timestamp — время создания сообщения.
- Hostname — узел, отправивший сообщение.
- Program или Tag — программа, которая создала событие.
- Message — текст события.

Facility, severity и тело Syslog-сообщения

16:9

Структура Syslog-сообщения (RFC 5424)

<13> 2024-01-15T10:30:45 web01 sshd[1234] Failed password for root from 192.168.1.100

PRI Timestamp Hostname Tag (Program + PID) Message Body

Facility (источник)

Код	Facility	Код	Facility
0	kern	7	news
1	user	8	uucp
2	mail	9	cron
3	daemon	10	authpriv
4	auth	11	ftp
5	syslog	16-23	local0-7
6	lpr	★	Пользовательские

Severity (важность)

0	1	2	3	4	5	6	7
Emergency	Alert	Critical	Error	Warning	Notice	Info	Debug
0-1: Система неработоспособна / Немедленное вмешательство		2-4: Критическое состояние / Ошибка / Предупреждение		5-7: Заметное событие / Информация / Отладка			

★ Чем выше значение — тем ниже важность

PRI = (Facility × 8) + Severity

Пример: Facility=1 (user), Severity=5 (notice) → PRI=13 → <13>

Пример разбора

Исходное сообщение

<13>2024-01-15T10:30:45 web01 sshd[1234]:

<13> → Facility=1 (user), Severity=5 (notice)

2024-01-15T10:30:45 → Timestamp (ISO 8601)

web01 → Hostname (отправитель)

sshd[1234] → Program + PID

Failed password for root... → Message Body

Ключевые моменты

1

Facility определяет **источник** события (kern, mail, cron, local0 и др.)
Severity определяет **важность** события (0 — Emergency, 7 — Debug)

2

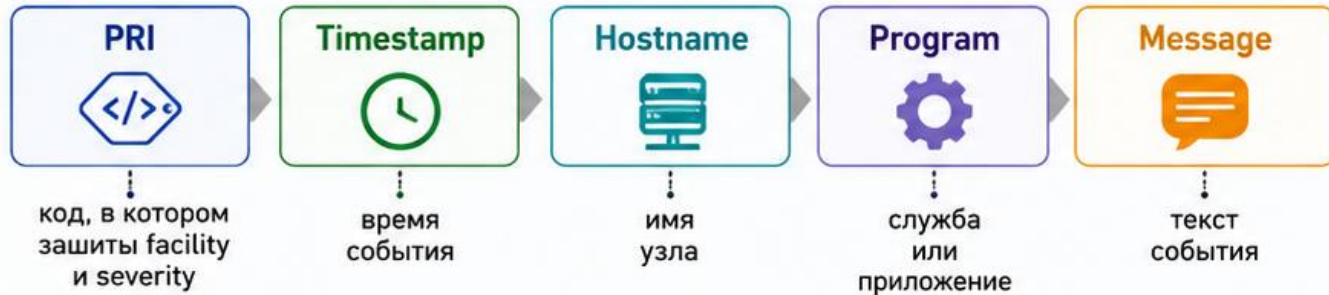
PRI вычисляется по формуле: **(Facility × 8) + Severity**
PRI помещается в угловые скобки: **<PRI>** в начале сообщения

★ Практическое применение: фильтрация по facility (auth — безопасность) и severity (Error — ошибки)
Правильная классификация помогает быстро находить критичные события в потоке логов

FACILITY, SEVERITY И ТЕЛО SYSLOG-СООБЩЕНИЯ

Из чего состоит Syslog-сообщение и как понять его важность

1 ЧТО ВХОДИТ В SYSLOG-СООБЩЕНИЕ



`>_ <34> Oct 11 10:15:22 web01 sshd: Failed password for admin from 192.168.10.15`

2 FACILITY — ИСТОЧНИК СООБЩЕНИЯ

Facility показывает, откуда пришло событие



auth/authpriv
аутентификация
и безопасность



cron
планирующих
заданий



daemon
системные
службы



kern
ядро
Linux



mail
почтовые
службы



local0–local7
пользовательские
приложения

3 SEVERITY — УРОВЕНЬ ВАЖНОСТИ

Severity показывает срочность реакции

КРИТИЧНО ↑ ↓ ОБЫЧНО	0	Emergency	система неработоспособна
	1	Alert	нужно немедленное действие
	2	Critical	критическая проблема
	3	Error	ошибка
	4	Warning	предупреждение
	5	Notice	значимое нормальное событие
	6	Informational	информационное сообщение
	7	Debug	отладка

4 КАК ЧИТАЕТСЯ PRI

$$\text{PRI} = \text{Facility} \times 8 + \text{Severity}$$

Facility (источник) **4** auth × **8** + Severity (важность) **2** critical = **34** PRI



Если facility = 4 (auth) и severity = 2 (critical), то $\text{PRI} = 4 \times 8 + 2 = 34$

5 КАК АДМИНИСТРАТОР ИСПОЛЬЗУЕТ ЭТУ ИНФОРМАЦИЮ



Фильтрует логи по источнику



Понимает важность события



Быстрее ищет инциденты



Настраивает пересылку и правила в rsyslog



Facility отвечает на вопрос:
ОТКУДА?



Severity отвечает на вопрос:
НАСКОЛЬКО ВАЖНО?



Message отвечает на вопрос:
ЧТО СЛУЧИЛОСЬ?

Facility: откуда пришло событие

- Facility помогает классифицировать источник события.
- auth и authpriv — авторизация и безопасность.
- cron — планировщик заданий.
- daemon — системные службы.
- kern — сообщения ядра.
- mail — почтовые службы.
- local0-local7 — пользовательские или учебные сценарии.

Severity: насколько событие важно

- Severity показывает срочность реакции администратора.
- 0 Emergency — система неработоспособна.
- 1 Alert — требуется немедленное вмешательство.
- 2 Critical и 3 Error — критическая ситуация или ошибка.
- 4 Warning — предупреждение.
- 5 Notice — значимое нормальное событие.
- 6 Informational и 7 Debug — информационные и отладочные сообщения.

Severity 0–7

Код	Название	Смысл
0	Emergency	Система неработоспособна
1	Alert	Нужно немедленное действие
2	Critical	Критическая проблема
3	Error	Ошибка
4	Warning	Предупреждение
5	Notice	Важное нормальное событие
6	Informational	Информация
7	Debug	Отладка

PRI: короткий код facility и severity

- PRI — число в начале Syslog-сообщения, которое кодирует facility и severity.
- Формула: $PRI = Facility \times 8 + Severity$.
- PRI записывается в угловых скобках.
- Пример: facility=user имеет код 1.
- severity=notice имеет код 5.
- $PRI = 1 \times 8 + 5 = 13$.
- В сообщении это будет выглядеть как <13>.

UDP 514 и TCP 514

- UDP 514 — простой и быстрый вариант без подтверждения доставки.
- TCP 514 использует соединение и надёжнее при передаче важных событий.
- UDP подходит для простых учебных сценариев.
- TCP предпочтительнее, если потеря логов недопустима.
- Firewall должен разрешать выбранный протокол и порт.
- В production для защиты канала дополнительно используют TLS.

Итог раздела 2

1

Syslog задаёт структуру сообщения и способ передачи событий.

2

Facility отвечает на вопрос: откуда событие.

3

Severity отвечает на вопрос: насколько событие важно.

4

UDP быстрее, но не гарантирует доставку.

5

TCP надёжнее и удобнее для важных журналов.



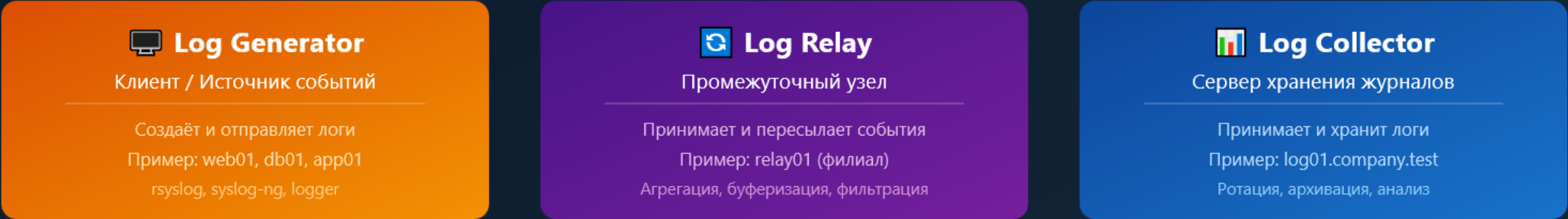
3. Участники процесса

Клиент, релей и сервер-коллектор

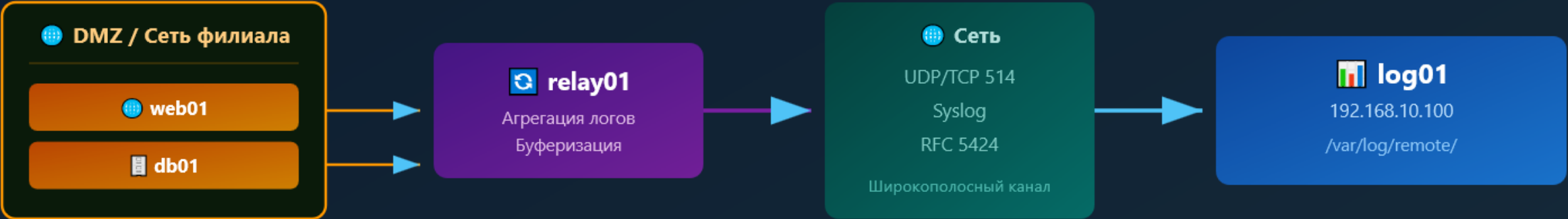
Три роли в Syslog-инфраструктуре

- Log Generator создаёт события и отправляет их наружу.
- Log Relay принимает события из сегмента и пересылает дальше.
- Log Collector принимает и хранит журналы от многих узлов.
- В простой лаборатории релей можно не использовать.
- В крупных сетях релей полезен для филиалов, DMZ и сегментов.

Клиент, релей и сервер-коллектор в Syslog-инфраструктуре



Топология с релем

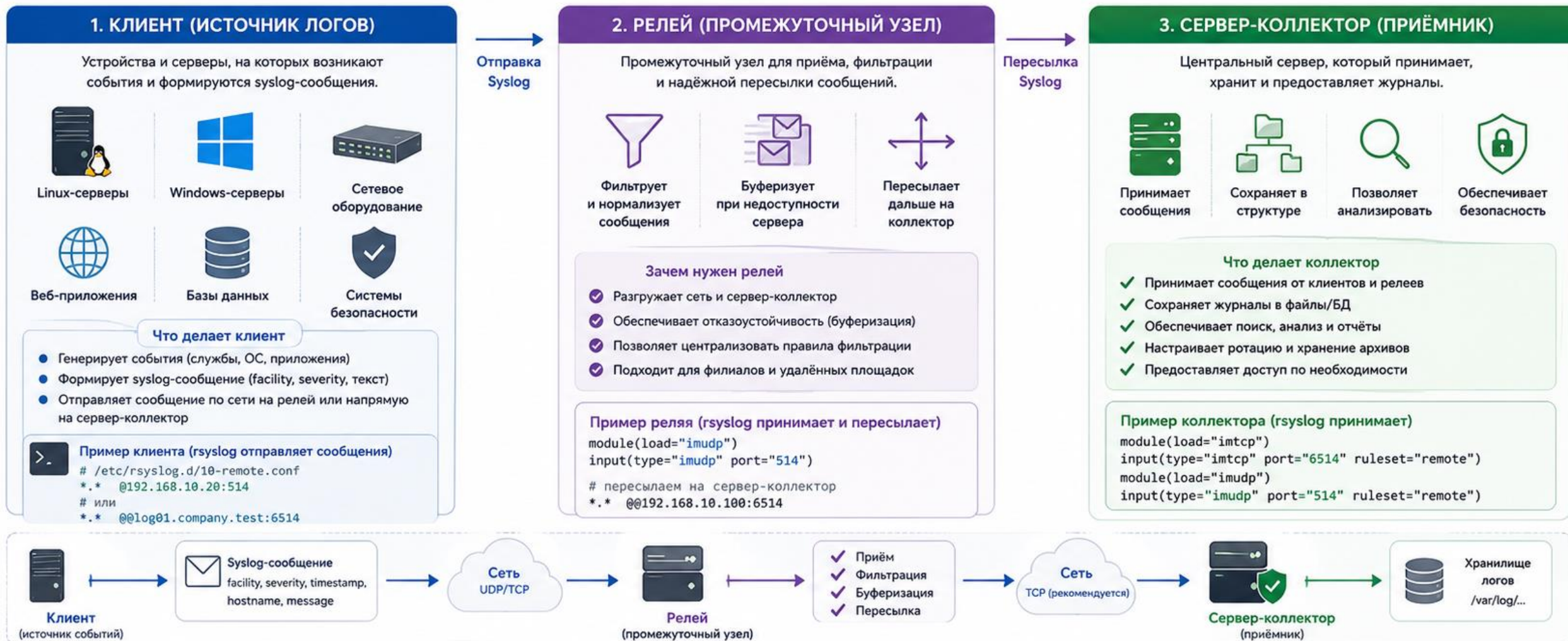


Сравнение ролей

Роль	Задача	Пример	Особенности	Транспорт
 Клиент (Generator)	Создаёт и отправляет системные события	web01, db01 app01, firewall	Может иметь очередь на случай недоступности	UDP/TCP 514 TLS (опционально)
 Релей (Relay)	Принимает, агрегирует и пересылает события	relay01 (филиал/сегмент)	Снижает число соединений Буферизация на relay	TCP (надёжный) TLS (защищённый)

КЛИЕНТ, РЕЛЕЙ И СЕРВЕР-КОЛЛЕКТОР В SYSLOG

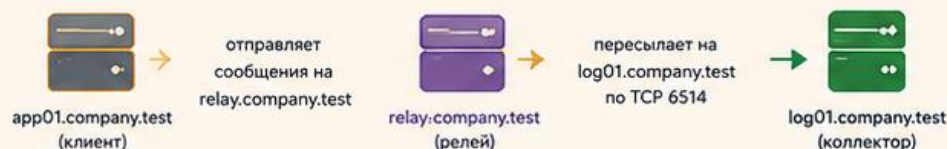
Как путешествуют системные журналы от источника до центра хранения



КАКИЕ ПРОТОКОЛЫ ИСПОЛЬЗУЕМ

- UDP/514** → Просто и быстро, но без гарантии доставки. Подходит для некритичных событий.
- TCP/514** → Надёжная доставка, но без шифрования.
- TLS/6514** → Надёжная доставка и шифрование. Рекомендуется для продакшена.

ПРИМЕР ПУТИ СООБЩЕНИЯ



ЧТО ВАЖНО ПОМНИТЬ

- ✓ Синхронизируйте время на всех узлах (NTP)
- ✓ Защитите каналы передачи (TLS, VPN, фаервол)
- ✓ Настройте ротацию и хранение журналов
- ✓ Контролируйте доступ к журналам

Централизация логов повышает видимость и безопасность инфраструктуры

Log Generator

- Log Generator — источник событий.
- Это может быть Linux-сервер, маршрутизатор, firewall, веб-сервер или приложение.
- На Linux роль отправителя часто выполняет rsyslog.
- Клиент должен знать адрес log-сервера.
- Клиент должен знать транспорт: UDP или TCP.
- В нашем стенде web01 отправляет события на 192.168.10.100.

Log Collector

- Log Collector — сервер хранения журналов.
- Принимает сообщения по сети.
- Раскладывает события по файлам.
- Хранит историю событий.
- Должен иметь место на диске и ротацию.
- Должен ограничивать доступ к журналам.
- В нашем стенде эту роль выполняет log01.

Log Relay

- Log Relay — промежуточный узел между источниками и центральным коллектором.
- Полезен в филиалах и сегментах сети.
- Снижает число прямых соединений к центральному серверу.
- Может буферизовать сообщения при недоступности канала.
- Может фильтровать или нормализовать события.
- В базовой лаборатории не используется, но важен для понимания enterprise-сценариев.

Итог раздела 3

- 1 Клиент создаёт и отправляет логи.
- 2 Коллектор принимает и хранит логи.
- 3 Релей нужен для более сложных сетей.
- 4 В лаборатории достаточно двух ролей: web01 как клиент и log01 как коллектор.

4. rsyslog: как сообщение проходит обработку

Input → Rules → Output

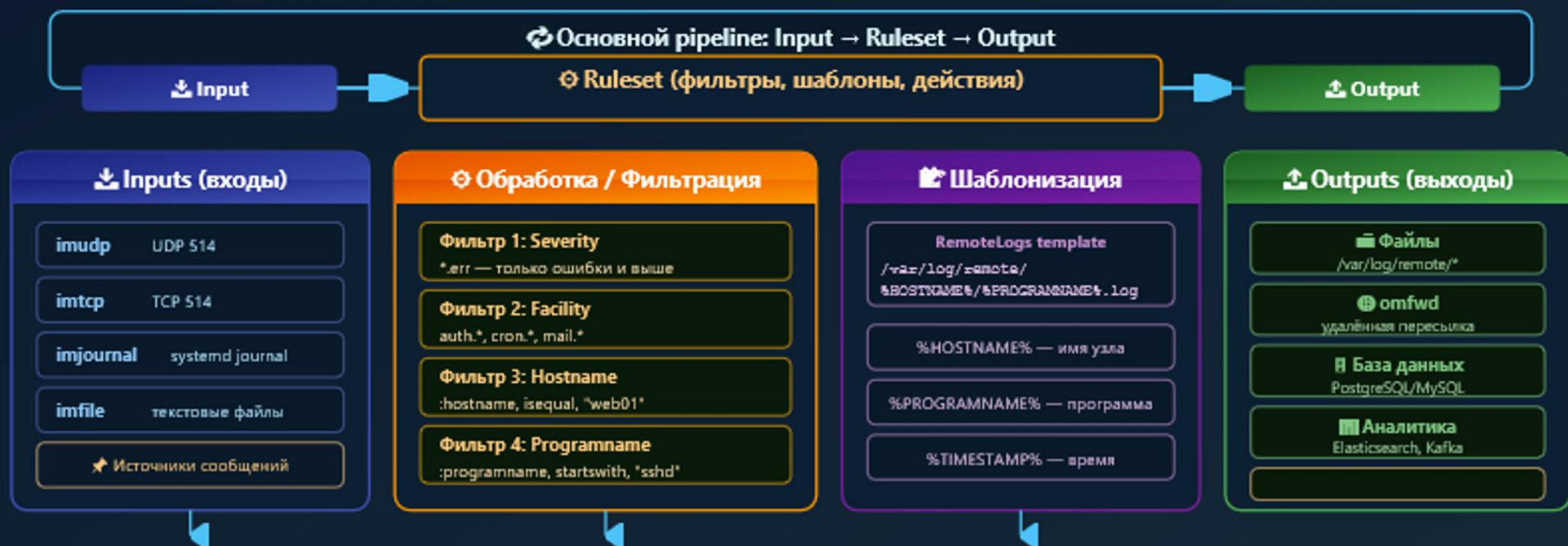
Что делает rsyslog

- rsyslog — служба обработки логов, которая принимает сообщения, применяет правила и записывает или пересылает результат.
- Принимает локальные сообщения из systemd journal или системных сокетов.
- Может принимать сетевые сообщения по UDP и TCP.
- Фильтрует сообщения по facility, severity, hostname, programname или тексту.
- Записывает события в файлы.
- Может пересылать сообщения на другой сервер.
- Поддерживает модули, шаблоны, очереди и TLS.

Главные файлы конфигурации

- `/etc/rsyslog.conf` — основной конфигурационный файл.
- `/etc/rsyslog.d/` — каталог для отдельных правил.
- Для лабораторных правил удобнее создавать отдельные файлы в `rsyslog.d`.
- После изменения конфигурации нужно проверить синтаксис.
- Затем службу нужно перезапустить или перечитать.

Pipeline обработки сообщений rsyslog



PIPELINE ОБРАБОТКИ СООБЩЕНИЙ RSYSLOG

Как сообщение проходит путь от источника до файла или удалённого сервера



КАК ЭТО РАБОТАЕТ

- 1 rsyslog принимает сообщения из локальных и сетевых источников
- 2 складывает их во внутреннюю очередь
- 3 применяет правила и фильтры
- 4 выполняет нужное действие: записывает или пересылает
- 5 при необходимости использует буферизацию и повторную отправку

ЧТО ВАЖНО АДМИНИСТРАТОРУ

- ✓ Проверять inputs и открытые порты
- ✓ Тестировать конфигурацию: `rsyslogd -N1`
- ✓ Разделять локальные и удалённые логи
- ✓ Использовать templates и stop
- ✓ Настраивать ротацию и контроль доступа



rsyslog = **входы** → **очередь** → **фильтрация** → **действие** → **хранение или пересылка**

Input, Rules, Output

- Input получает сообщение из файла, сокета, сети или модуля.
- Rules фильтруют сообщение и выбирают действие.
- Templates задают формат или путь хранения.
- Output записывает сообщение в файл или пересылает его.
- Такая цепочка одинаково работает для локальных и удалённых событий.

Модули imudp, imtcp и omfwd

- imudp — приём Syslog-сообщений по UDP.
- imtcp — приём Syslog-сообщений по TCP.
- omfwd — пересылка сообщений на удалённый сервер.
- Модули нужно включать явно.
- Если модуль не загружен, соответствующий вход или выход работать не будет.

Итог раздела 4

- 1 **rsyslog обрабатывает сообщение через цепочку input → rules → output.**
- 2 **Отдельные файлы в /etc/rsyslog.d/ удобнее сопровождать.**
- 3 **Синтаксис нужно проверять до перезапуска службы.**
- 4 **Модули включают сетевой приём и пересылку.**

5. Практика: настройка log01 как коллектора

log01 принимает и сохраняет удалённые журналы

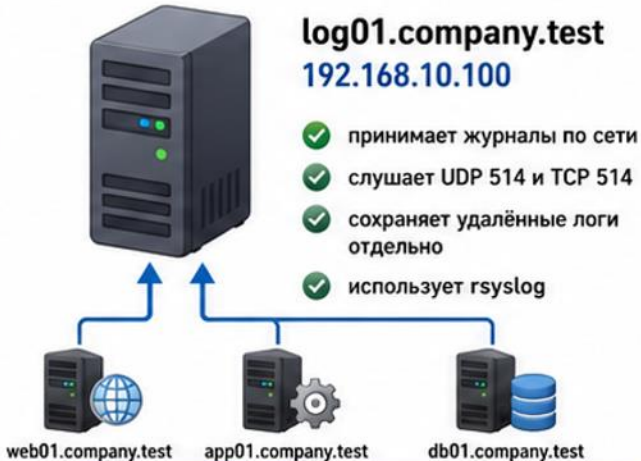
Задача log01

- log01 должен принимать сообщения от удалённых узлов и сохранять их отдельно от собственных локальных журналов.
- Включить приём Syslog по UDP и TCP 514.
- Создать каталог /var/log/remote.
- Назначить права для записи rsyslog.
- Создать шаблон RemoteLogs.
- Сохранять удалённые логи по имени хоста и программы.
- Остановить дальнейшую обработку удалённых сообщений через & stop.

НАСТРОЙКА log01 КАК RSYSLOG-КОЛЛЕКТОРА

Как сервер принимает удалённые Syslog-сообщения и сохраняет их по хостам и программам

1 РОЛЬ СЕРВЕРА log01



2 1. ПОДГОТОВКА КАТАЛОГА

```
$ sudo mkdir -p /var/log/remote
```

→ создать каталог для удалённых журналов

```
$ sudo chown syslog:adm /var/log/remote
```

→ назначить владельца для записи rsyslog

```
$ sudo chmod 750 /var/log/remote
```

→ ограничить доступ к логам



/var/log/remote

3 2. ВКЛЮЧЕНИЕ ПРИЁМА ПО СЕТИ



/etc/rsyslog.d/10-remote-input.conf

```
module(load="imudp")  
input(type="imudp" port="514")
```

```
module(load="imtcp")  
input(type="imtcp" port="514")
```

✓ imudp — приём Syslog по UDP

✓ imtcp — приём Syslog по TCP

✓ порт 514 должен быть открыт в firewall

4 3. ШАБЛОН ХРАНЕНИЯ REMOTELOGS



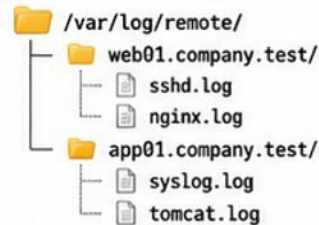
/etc/rsyslog.d/20-remote-store.conf

```
$template RemoteLogs,"/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"  
*. * ?RemoteLogs  
& stop
```

%HOSTNAME% → имя узла

%PROGRAMNAME% → имя программы

& stop → не даёт сообщению попасть в другие локальные правила



5 4. ПРИМЕНЕНИЕ И ПРОВЕРКА

```
$ sudo rsyslogd -N1
```



проверить синтаксис конфигурации

```
$ sudo systemctl restart rsyslog
```



перезапустить службу

```
$ sudo ss -tulnp | grep ':514'
```



убедиться, что сервер слушает порт 514

Netid	State	Recv-Q	Send-Q	Local Address:Port	Process
udp	UNCONN	0	0	0.0.0.0:514	users:("rsyslogd",pid=1234,fd=6)
tcp	LISTEN	0	128	0.0.0.0:514	users:("rsyslogd",pid=1234,fd=7)

6 ЧТО ВАЖНО НЕ ЗАБЫТЬ



открыть UDP/TCP 514 в firewall



хранить удалённые логи отдельно от локальных



использовать & stop, чтобы избежать дублирования



следить за местом на диске и logrotate



ограничить доступ к журналам



Syslog
UDP/TCP 514



log01
(rsyslog)



/var/log/remote/
%HOSTNAME%/
%PROGRAMNAME%.log

Каталог для удалённых логов

BASH

```
sudo mkdir -p /var/log/remote  
sudo chown syslog:adm /var/log/remote  
sudo chmod 750 /var/log/remote
```

Что важно

- mkdir создаёт корневой каталог для удалённых журналов.
- syslog:adm даёт службе rsyslog право записи.
- 750 ограничивает доступ к журналам.

Приём UDP и TCP на log01

BASH

```
sudo nano /etc/rsyslog.d/10-remote-input.conf
```

Что важно

- Файл создаётся отдельно, чтобы не изменять основной rsyslog.conf.
- В этом файле включаются сетевые входы imudp и imtcp.
- UDP/TCP 514 должны быть разрешены firewall.

Содержимое 10-remote-input.conf

TEXT

```
module(load="imudp")  
input(type="imudp" port="514")
```

```
module(load="imtcp")  
input(type="imtcp" port="514")
```

Что важно

- imudp включает приём UDP-сообщений.
- imtcp включает приём TCP-сообщений.
- port="514" задаёт стандартный Syslog-порт.

Шаблон хранения RemoteLogs

BASH

```
sudo nano /etc/rsyslog.d/20-remote-store.conf
```

Что важно

- Этот файл отвечает за путь хранения удалённых сообщений.
- Шаблон использует имя хоста и имя программы.
- Так логи разных серверов не смешиваются в одном файле.

Содержимое 20-remote-store.conf

TEXT

```
$template RemoteLogs, "/var/log/remote/%HOSTNAME%/%PROGRAMNAME%.log"  
*.* ?RemoteLogs  
& stop
```

Что важно

- %HOSTNAME% подставляет имя отправителя.
- %PROGRAMNAME% подставляет программу-источник.
- *.* означает все facility и все severity.
- & stop не даёт удалённым сообщениям дублироваться в локальных логах log01.

Зачем нужен & stop

- rsyslog обрабатывает правила сверху вниз.
- Без stop удалённое сообщение может попасть и в /var/log/remote, и в локальные файлы log01.
- Это создаёт дублирование и усложняет анализ.
- Для коллектора важно отделять чужие события от собственных.
- & stop завершает обработку текущего сообщения после записи в RemoteLogs.

Проверка конфигурации log01

BASH

```
sudo rsyslogd -N1  
sudo systemctl restart rsyslog  
sudo ss -tulnp | grep ':514'
```

Что важно

- rsyslogd -N1 проверяет синтаксис без запуска.
- restart применяет изменения.
- ss показывает, слушает ли сервер порт 514.
- Если порт не слушается, клиентские сообщения не придут.

Итог настройки log01

1

log01 принимает UDP и TCP Syslog-сообщения.

2

Удалённые логи сохраняются в /var/log/remote.

3

Путь строится по hostname и programname.

4

& stop отделяет удалённые события от локальных логов коллектора.

5

После настройки нужно проверить синтаксис, службу и открытые порты.

6. Практика: настройка web01 как клиента

web01 пересылает события на log01

Задача web01

- Создавать обычные локальные системные события.
- Передавать события на log01.
- Выбрать транспорт: UDP или TCP.
- В простом варианте отправлять все события через *.*.
- В production лучше фильтровать нужные facility и severity.
- Для надёжности лучше использовать TCP и очередь.

ПЕРЕСЫЛКА ЛОГОВ WEB01 НА LOG01

Настраиваем rsyslog-клиент web01 для отправки логов на сервер-коллектор log01



1. УСЛОВИЯ



web01 — отправитель (клиент)
192.168.10.11



log01 — получатель (коллектор)
192.168.10.100



Протокол: UDP или TCP
Порт: 514



Синхронизированное время (NTP)
на обоих узлах

2. ПРОВЕРЯЕМ СЕТЕВОЕ СОЕДИНЕНИЕ

Проверяем, что web01 может достигать до log01.

Проверка UDP 514:

```
web01:~# nc -zuv 192.168.10.100 514
Connection to 192.168.10.100 514 port [udp/*] succeeded!
```

Проверка TCP 514:

```
web01:~# nc -zv 192.168.10.100 514
Connection to 192.168.10.100 514 port [tcp/*] succeeded!
```

3. РЕЗЕРВНАЯ КОПИЯ КОНФИГУРАЦИИ

Перед изменениями создаём резервную копию.

```
web01:~# sudo cp /etc/rsyslog.conf \
/etc/rsyslog.conf.bak.${date +%F_%H%M}
web01:~# sudo cp -r /etc/rsyslog.d \
/etc/rsyslog.d.bak.${date +%F_%H%M}
```

4. НАСТРАИВАЕМ ОТПРАВКУ ЛОГОВ НА LOG01

- 1 Открываем или создаём файл конфигурации отправки.

```
web01:~# sudo nano /etc/rsyslog.d/10-forward-to-log01.conf
```

- 2 Добавляем правила пересылки.

Отправим все сообщения (facility.*) на log01 по UDP.

```
# 10-forward-to-log01.conf
# Отправлять ВСЕ сообщения на log01 (UDP)
*,* @192.168.10.100:514

# (Альтернатива: по TCP - надёжнее, но медленнее)
# *,* @@192.168.10.100:514
```



Пояснения:

- *,* — все facility и все уровни severity
- @ — передача по UDP
- @@ — передача по TCP

- 3 (Опционально) Исключаем локальные записи, чтобы не дублировать их на log01.

Создаём файл исключений.

```
web01:~# sudo nano /etc/rsyslog.d/11-local-exclude.conf

# 11-local-exclude.conf
# Не отправлять локальные логи на сам localhost
if $hostname == 'web01.company.test' then stop
```

- 4 Проверяем синтаксис конфигурации.

```
web01:~# sudo rsyslogd -N1
rsyslogd: version ..., configuration OK
```

5. ПРИМЕНЯЕМ И ПРОВЕРЯЕМ

- 1 Перезапускаем rsyslog на web01.

```
web01:~# sudo systemctl restart rsyslog
```

- 2 Проверяем статус службы.

```
web01:~# systemctl status rsyslog
Active: active (running)
```

- 3 Генерируем тестовое сообщение.

```
web01:~# logger -p local0.info "Test from web01"
```



Проверяем на log01

На сервере-коллекторе сообщение должно появиться в файле удалённых логов.

```
log01:~# sudo tail -f /var/log/remote/web01.company.test/*.log
May 19 12:34:56 web01.company.test user: Test from web01
```

6. ПОЛЕЗНЫЕ КОМАНДЫ

- Посмотреть, куда будет отправлено сообщение:
web01:~# rsyslogd -N1
- Посмотреть очередь rsyslog:
web01:~# sudo rsyslogd -q
- Посмотреть активные правила:
web01:~# sudo rsyslogd -M
- Журнал службы rsyslog:
web01:~# journalctl -u rsyslog -f



7. СТРУКТУРА ФАЙЛОВ НА LOG01 (РЕЗУЛЬТАТ)

Логи web01 будут сохраняться в:

```
/var/log/remote/
├─ web01.company.test/
│  ├─ auth.log
│  ├─ syslog.log
│  ├─ cron.log
│  └─ local0.log # (например, сюда попадёт наше тестовое сообщение)
```



Преимущества:

- ✓ Централизация журналов
- ✓ Единое место хранения
- ✓ Удобный поиск и анализ
- ✓ Защита от потери логов при сбое узла

8. ВАЖНО ПОМНИТЬ

- ✓ Время на узлах должно быть синхронизировано (NTP)
- ✓ Для полной надёжности используйте TCP (@@)
- ✓ Не отправляйте логи на самовлано (stop), чтобы избежать циклов
- ✓ Следите за доступным местом на log01
- ✓ Регулярно проверяйте доставку (тест logger)



9. ВОЗМОЖНЫЕ ПРОБЛЕМЫ

- ✗ Нет соединения: проверьте сеть, порт 514 и фаервол
- ✗ Логи не приходят: проверьте конфиг, синтаксис (-N 1), статус rsyslog и тест logger
- ✗ Дублирование локальных логов: добавьте правило stop
- ℹ Потеря сообщений (UDP): используйте TCP для важных логов

Традиционный синтаксис пересылки

- *. * @192.168.10.100:514 отправляет все сообщения по UDP.
- *. * @@192.168.10.100:514 отправляет все сообщения по TCP.
- Один символ @ означает UDP.
- Два символа @@ означают TCP.
- Правило можно поместить в /etc/rsyslog.d/60-forward.conf.

Простая пересылка по TCP

BASH

```
sudo nano /etc/rsyslog.d/60-forward.conf
```

Что важно

- Файл создаётся на web01.
- Он описывает, куда web01 должен отправлять сообщения.
- Для учебного стенда используем TCP как более надёжный транспорт.

Содержимое 60-forward.conf

ТЕХТ

```
*.* @@192.168.10.100:514
```

Что важно

- *.* отправляет все facility и severity.
- @@ означает TCP.
- 192.168.10.100 — адрес log01.
- 514 — порт Syslog.

Современный action omfwd

- omfwd — модуль rsyslog для пересылки сообщений на удалённый сервер.
- RainerScript-синтаксис удобнее расширять.
- Можно явно указать target, port и protocol.
- Легче добавить очереди и повторные попытки.
- Удобнее переходить к TLS-передаче.
- Для базового занятия достаточно знать оба варианта.

Пересылка через omfwd

TEXT

```
action(  
  type="omfwd"  
  target="192.168.10.100"  
  port="514"  
  protocol="tcp"  
)
```

Что важно

- target — адрес log01.
- protocol="tcp" выбирает TCP.
- Такой синтаксис удобнее для production-настроек.
- Для простой лаборатории можно использовать @@.

Буферизация на клиенте

- Буферизация уменьшает потерю логов, если log01 временно недоступен.
- `queue.type="LinkedList"` включает очередь.
- `queue.filename` задаёт имя файлового буфера.
- `action.resumeRetryCount="-1"` включает бесконечные повторные попытки.
- `queue.saveonshutdown="on"` сохраняет очередь при остановке службы.

Проверка клиента web01

BASH

```
sudo rsyslogd -N1  
sudo systemctl restart rsyslog  
logger -p local0.warn "test from web01"
```

Что важно

- rsyslogd -N1 проверяет синтаксис клиентской конфигурации.
- restart применяет правило пересылки.
- logger создаёт тестовое Syslog-сообщение.
- local0.warn задаёт facility local0 и severity warning.

Проверка на log01

BASH

```
sudo find /var/log/remote -type f  
sudo tail -f /var/log/remote/web01/logger.log
```

Что важно

- find показывает, появился ли файл удалённого журнала.
- tail -f позволяет увидеть новые сообщения.
- Если файл не появился, нужно проверить сеть, порт 514 и правила rsyslog.

Итог настройки web01

- 1 web01 отправляет логи на log01.
- 2 Для UDP используется @, для TCP — @@.
- 3 TCP надёжнее для важных сообщений.

- 4 logger нужен для контролируемого теста.
- 5 На log01 сообщение должно появиться в /var/log/remote.



7. logger и logrotate

Как тестировать и не переполнить диск

logger: ручное тестовое сообщение

- `logger` — консольная утилита для отправки сообщения в систему логирования.
- Полезна для проверки пересылки.
- Может использоваться в Bash-скриптах.
- Позволяет явно задать `facility` и `severity`.
- Текст в кавычках становится телом Syslog-сообщения.
- На `log01` сообщение должно попасть в файл программы `logger`.

Пример logger

BASH

```
logger -p local0.warn "Тестовое предупреждение: сбой резервного копирования"
```

Что важно

- -p задаёт пару facility.severity.
- local0 — пользовательская facility.
- warn — severity warning.
- Текст в кавычках будет сохранён как message.

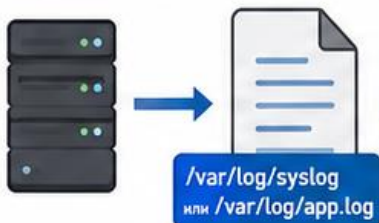
Зачем нужна ротация логов

- Журналы постоянно растут.
- На центральном коллекторе рост особенно быстрый.
- Без ротации можно переполнить диск.
- logrotate переименовывает старые файлы.
- Старые журналы можно сжимать gzip.
- Старые архивы можно удалять по политике хранения.

ЖИЗНЕННЫЙ ЦИКЛ ФАЙЛА ЖУРНАЛА И LOGROTATE

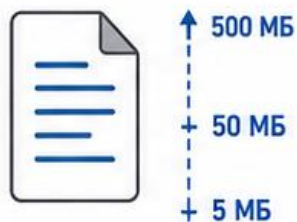
Как журналы растут, ротируются, архивируются и удаляются

1 Служба пишет в активный лог



- rsyslog, nginx, sshd и другие службы пишут в текущий файл
- Файл постоянно растёт по мере появления событий

2 Файл увеличивается



- Без контроля журналы занимают всё больше места
- На сервере-коллекторе рост особенно быстрый

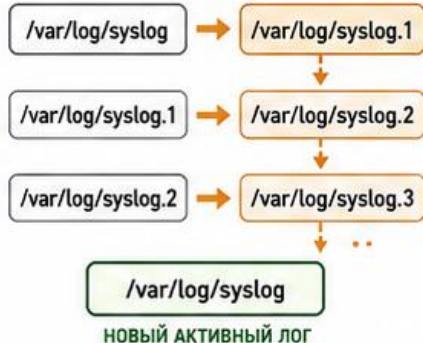
3 Срабатывает logrotate



- по расписанию: **daily / weekly / monthly**
- или по условию размера

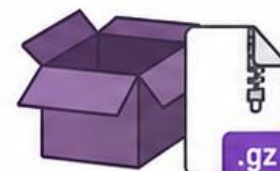
```
$ sudo logrotate /etc/logrotate.conf
```

4 Текущий файл ротируется



- текущий файл переименовывается
- создаётся новый активный лог
- служба продолжает писать уже в новый файл

5 Архивирование и сжатие



`/var/log/syslog.2.gz`
`/var/log/syslog.3.gz`
...

- старые журналы можно сжимать
- **compress** экономит место на диске

6 Удаление старых архивов



- хранятся только последние N копий
- старые архивы удаляются автоматически
- пример: **rotate 14**



активный лог



рост



ротация



новый файл



архив



удаление

ОСНОВНЫЕ ПАРАМЕТРЫ LOGROTATE



daily / weekly / monthly — период ротации



rotate 14 — число архивов



compress — сжатие



missingok — не считать ошибкой отсутствие файла



create — создать новый файл после ротации



postrotate — выполнить команду после ротации

ПОЧЕМУ ЭТО ВАЖНО



защита от переполнения диска



упорядоченное хранение журналов



удобство анализа и аудита



стабильная работа служб логирования

ПРОВЕРКА:



```
logrotate -d /etc/logrotate.d/rsyslog  
— тест без изменений
```



```
logrotate -f /etc/logrotate.d/rsyslog  
— принудительная ротация
```

Основные параметры logrotate

- daily, weekly, monthly — период ротации.
- rotate 14 — хранить 14 архивных копий.
- compress — сжимать старые журналы.
- missingok — не считать ошибкой отсутствие файла.
- postrotate — выполнить команду после ротации.
- После ротации важно убедиться, что rsyslog пишет в новый файл.

Проверка logrotate

BASH

```
sudo logrotate -d /etc/logrotate.d/rsyslog  
sudo logrotate -f /etc/logrotate.d/rsyslog
```

Что важно

- -d запускает dry-run без реальных изменений.
- -f принудительно выполняет ротацию.
- После теста проверьте, что rsyslog продолжает писать в актуальный лог.
- На collector-сервере ротация критична для защиты от переполнения диска.

Итог раздела 7

1 **logger** создаёт тестовые сообщения.

4 Сжатие уменьшает объём старых журналов.

2 **logrotate** защищает сервер от бесконтрольного роста логов.

5 Проверку ротации нужно выполнять осторожно.

3 Ротация разделяет текущий файл и архивы.



8. Диагностика и типовые ошибки

Где искать проблему: клиент, сеть или сервер

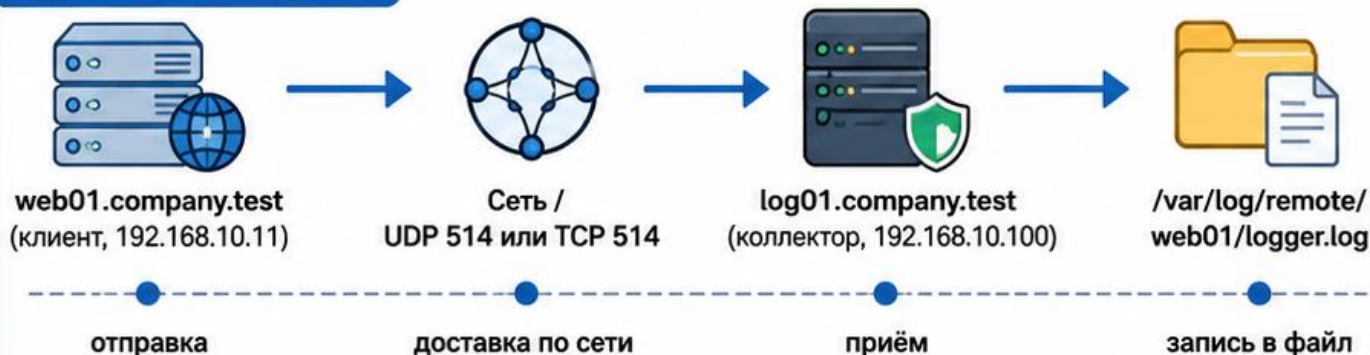
Правильная логика диагностики

- Диагностика начинается с разделения проблемы: клиент не отправляет, сеть не пропускает или сервер не принимает.
- Сначала проверить синтаксис конфигурации rsyslog.
- Затем проверить статус службы.
- Потом проверить порт 514 на log01.
- После этого отправить тест через logger.
- На log01 проверить файл в /var/log/remote.
- Если сообщения нет, посмотреть journalctl и права каталогов.

ДИАГНОСТИКА ДОСТАВКИ SYSLOG-СООБЩЕНИЯ

Как проверить путь сообщения от клиента до сервера-коллектора.

1. ПУТЬ СООБЩЕНИЯ



2. ПРОВЕРКА НА КЛИЕНТЕ web01



- ✓ проверить синтаксис:
`sudo rsyslogd -N1`
- ✓ проверить службу:
`systemctl status rsyslog`
- ✓ проверить правило пересылки:
`*.* @@192.168.10.100:514`
- ✓ отправить тест:
`logger -p local0.warn "test from web01"`

если ошибка синтаксиса — сообщение не уйдёт.

3. ПРОВЕРКА СЕТИ



- на сервере проверить прослушивание:
`ss -tulnp | grep ':514'`
- с клиента проверить TCP:
`nc -vz 192.168.10.100 514`
- проверить firewall / ufw / nftables
- убедиться, что клиент и сервер используют один и тот же транспорт: UDP или TCP



порт закрыт



не тот протокол

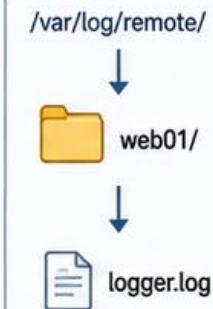


блокировка firewall

4. ПРОВЕРКА НА КОЛЛЕКТОРЕ log01



- проверить syntax и службу rsyslog
`sudo rsyslogd -N1, systemctl status rsyslog`
- проверить наличие каталога:
`/var/log/remete/web01/`
- смотреть файл в реальном времени:
`tail -f /var/log/remote/web01/logger.log`
- если сообщение попало в локальный /var/log/syslog, проверить шаблон RemoteLogs и правило «& stop»



5. ГДЕ ИСКАТЬ ПРИЧИНУ

Симптом	Что проверить
Сообщение не отправляется	конфиг клиента, rsyslogd -N1, status rsyslog
Порт недоступен	ss, nc, firewall, маршрут
Сообщение пришло, но не в тот файл	template RemoteLogs, %HOSTNAME%, %PROGRAMNAME%
Сообщение дублируется	правило & stop
Логи иногда теряются	UDP, перегрузка, лучше TCP и очередь

АЛГОРИТМ ДИАГНОСТИКИ



СОВЕТ: Лучше использовать понятное тестовое сообщение, например: `logger -p local0.warn "test from web01"`

Проверка синтаксиса и службы

BASH

```
sudo rsyslogd -N1  
systemctl status rsyslog  
journalctl -u rsyslog -n 50 --no-pager
```

Что важно

- rsyslogd -N1 проверяет конфигурацию до перезапуска.
- systemctl status показывает, запущена ли служба.
- journalctl показывает последние ошибки rsyslog.
- Ошибка в одном конфигурационном файле может сломать всю службу.

Проверка сети и портов

BASH

```
sudo ss -tulnp | grep ':514'  
nc -vz 192.168.10.100 514
```

Что важно

- ss выполняется на log01 и показывает, слушается ли порт 514.
- nc выполняется на клиенте и проверяет TCP-доступность.
- Для UDP проверяют firewall и факт появления сообщения.
- Если firewall включён, нужно разрешить выбранный протокол и порт.

Проверка доставки

BASH

```
logger -p local0.warn "delivery test from web01"  
sudo tail -f /var/log/remote/web01/logger.log
```

Что важно

- logger создаёт понятное тестовое сообщение.
- tail -f на log01 показывает, пришло ли оно.
- Если каталог с именем хоста не появился, проверьте hostname и шаблон RemoteLogs.
- Если сообщение попало в /var/log/syslog, проверьте правило & stop.

Типовые ошибки

Ошибка	Что проверить
Забыли restart rsyslog	<code>systemctl restart rsyslog</code> после правок
Firewall блокирует 514	Разрешить UDP/TCP 514
Клиент отправляет UDP, сервер слушает только TCP	Согласовать транспорт
Нет & stop	Проверить <code>20-remote-store.conf</code>
Нет прав на <code>/var/log/remote</code>	<code>chown syslog:adm</code> и <code>chmod 750</code>
Сообщение не появилось	<code>logger</code> , <code>tail</code> , <code>journalctl</code> , <code>ss</code> , <code>nc</code>
Диск заполняется	Настроить <code>logrotate</code>
Неверный путь файла	Проверить <code>%HOSTNAME%</code> и <code>%PROGRAMNAME%</code>

Итог диагностики

1

Диагностику нужно вести по слоям:
конфигурация → служба → сеть →
файл.

2

logger даёт контролируемое тестовое
сообщение.

3

rsyslogd -N1 нужно запускать перед
перезапуском службы.

4

ss и nc помогают отделить сетевую
проблему от проблемы rsyslog.

5

journalctl показывает ошибки службы.

6

logrotate защищает collector от
переполнения диска.

Итоги лекции

1

Централизованный сбор логов повышает наблюдаемость инфраструктуры.

2

Syslog классифицирует события через facility и severity.

3

rsyslog реализует приём, фильтрацию, запись и пересылку логов.

4

log01 принимает и хранит удалённые логи.

5

web01 отправляет события на log01.

6

logger используется для тестирования доставки.

7

logrotate необходим для контроля роста журналов.

8

Диагностика начинается с разделения проблемы: клиент, сеть или сервер.

Контрольные вопросы

- 1 Почему локальные журналы недостаточны для инфраструктуры из многих серверов?
- 2 Что означает facility в Syslog-сообщении?
- 3 Что означает severity?
- 4 Чем UDP 514 отличается от TCP 514?
- 5 Какие роли выполняют log generator, relay и collector?
- 6 Зачем используется `/etc/rsyslog.d/`?
- 7 Для чего нужны `imudp` и `imtcp`?
- 8 Зачем в шаблоне `RemoteLogs` используются `%HOSTNAME%` и `%PROGRAMNAME%`?
- 9 Для чего нужен `& stop`?
- 10 Чем отличаются `@` и `@@` в правилах пересылки?
- 11 Как отправить тестовое сообщение через `logger`?
- 12 Что проверить, если сообщение не появилось в `/var/log/remote`?